# Context Tape: Observing, Costing and Controlling Context Growth in Multi-Turn LLM Agents

Ashish Ranjan Sharma

Technical report • Version 1.4 • October 2026

Companion to *PIR* (v2.4)

**Abstract**

PIR looked at a single prompt. This report looks at what happens to that prompt over a conversation. In a multi-turn agent the context window is rebuilt and resent on every model call. The fixed prefix (instructions, tool definitions, skills, memory) is joined by user messages, assistant messages and tool results, which grow until the agent compacts them into a summary. We propose the *Context Tape*, a per-turn record and visualisation of this process. It borrows the categories of Claude Code's `/context` view, maps them to a framework-neutral set, and draws them as a coloured stacked histogram over turns, with an event strip for tool calls, memory reads and writes, and compactions, and with a volume panel of cached input, uncached input and output tokens, in the style of a stock ticker. Under the volume panel sits an *evaluation tape*: per-turn results of code checks, LLM judges and user signals, aggregated by family so that tens or hundreds of checks stay readable, and aligned with the context that each turn was answered from. We define the metrics the tape makes visible: occupancy, growth per turn, headroom, call amplification, input-to-output ratio, cache retention and compaction ratio, and on the evaluation side, fail rate per family, first failing turn, pressure gap, post-compaction dip and judge coverage. We give a simple cost model. Without compaction, billed input grows quadratically with the number of turns. With threshold compaction it becomes a sawtooth that is linear on average. The best threshold then follows the same square-root law as the economic order quantity in inventory theory. We describe the instrumentation needed to draw the tape from OpenTelemetry spans that the Microsoft Agent Framework already emits, collected in MLflow and grouped by session, with one record per model call so that multi-agent sessions get a lane per agent and model. Only token counts per category and evaluation results are added, never prompt text. On two synthetic sessions (a coding agent with a 200k window and a banking assistant with 128k), billed input exceeds output by 102:1 and 168:1. About 90% of input is served from the prompt cache. Uncached tokens, which drive both cost and time to first token, come mainly from compaction and from memory writes placed in the prompt prefix. Moving memory out of the prefix cuts uncached input by 13–29%. Capping tool results cuts it by 27–31%. Turning compaction off doubles billed input. The square-root model predicts the cheapest compaction threshold within one sweep step for both sessions. To decide whether a new version of the prompt is better, we run the same multi-turn scenarios, with recorded tool responses, on both versions several times, score each run with a three-layer rubric (gates, a weighted quality score, cost diagnostics), and compare per scenario with a bootstrap interval calibrated by an A/A run. A diff tape shows which turns changed, and a champion–challenger loop promotes a version only if it is better, or cheaper and no worse, and is not contradicted on held-out scenarios. On a synthetic suite the loop rejects a change that fixes its target rule but breaks another, promotes a PIR refactor on cost, promotes a fix for knowledge lost at compaction, and rejects an optimiser rewrite that overfits. An interactive page, an MLflow exporter and all scripts accompany the report.

## 1 Introduction

A chat agent does not have a prompt; it has a sequence of prompts. Each model call sends the whole context again: the system prompt, the tool schemas, any loaded skills and memory, every earlier message, and every earlier tool result. A turn that calls three tools makes four model calls and sends the context four times. Over a session the context grows, the bill grows faster, and at some point the agent must compact: replace the history with a summary and start growing again.

Engineers rarely see this process. Logs show single requests. Dashboards show totals per day. The one widely used view of context composition, Claude Code's `/context` command [3], shows a snapshot of the current window as a grid of coloured squares, but not how it got there. In our own work on a three-tier chat agent with a 25-second response target [2], the questions we could not answer quickly were about trajectories. When does a session first compact? What fills the window just before it does? Why does a turn after a memory write take longer than the turn before it? How much of the input do we pay for at the full rate? And, once cost is understood: did the answers get worse as the window filled, or just after it was compacted?

The Context Tape answers these questions with one picture and a small set of numbers. Its contributions are:

1. **A record.** A per-turn record of context composition by category, billed input (cached and uncached), output, model calls, tool calls, memory events and compactions, with one row per model call when several agents or models share a session (Section 3, Appendix A).
2. **A visualisation.** A coloured, step-shaped stacked histogram of the window over turns, aligned with an event strip, a volume panel and an evaluation tape, plus the `/context` grid for any selected turn (Figures 1 and 2).
3. **Metrics and a cost model.** Definitions of occupancy, growth, headroom, amplification, cache retention and compaction ratio. An analysis showing quadratic growth without compaction and a square-root rule for the cost-optimal compaction threshold (Sections 4 and 5).
4. **Instrumentation.** A mapping from OpenTelemetry GenAI spans, as emitted by the Microsoft Agent Framework (MAF) [4] and stored in MLflow [6], to the record, with privacy-safe category attributes and an exporter (Section 6).
5. **An evaluation tape.** A way to collect per-turn evaluation results from code checks, LLM judges and user signals, join them to the record, and draw them under the volume panel so that hundreds of checks reduce to a summary bar, a handful of family lanes and a failures-only drill-down (Section 7, Figures 1 and 4).
6. **A comparison loop for prompt versions.** A fixed multi-turn suite with tool fixtures, a rubric of prompt quality, a paired comparison with a decision rule, a diff tape, and a champion–challenger loop that keeps the best version so far (Section 8, Figures 5 and 6).
7. **An evaluation of the cost side.** Two synthetic sessions, four policy ablations and a threshold sweep that checks the model's prediction (Section 9).

The tape is the time axis that PIR [1] lacked. PIR's Prompt Stack orders a single prompt by concern layer and scope envelope so that the cacheable prefix is as long as possible. The tape shows whether that prefix survives a real session, and the evaluation tape shows whether the rules PIR extracted from the prompt are still followed as the session goes on. The comparison loop closes the circle: a PIR refactor or rewrite of the prompt is adopted only if the tapes of the same scenarios say it is better, or cheaper and no worse.

## 2 Background

**What is in the window.** Claude Code's `/context` view splits the window into System prompt, System tools, MCP tools, Custom agents, Memory files, Skills, Messages, Free space and an Autocompact buffer [3]. The buffer is space kept free so that the compaction call itself fits. Most agent frameworks have the same parts under other names. MAF, for example, has agent instructions, function tools, context providers and a chat history with optional reducers [4].

**How it is billed.** Every call is billed for its full input. A turn with $k$ calls pays for the context $k$ times, each time a little larger. Provider prompt caches reuse the longest previously seen prefix of at least 1,024 tokens and charge cached tokens at a fraction of the normal rate, around 10% for current OpenAI models [8]. The cache is keyed on an exact prefix. Any change near the start of the prompt, such as a new memory line in the system message or a compaction that rewrites history, turns the rest of the prompt into uncached input [9, 10]. Uncached input also sets the prefill time and therefore the time to first token, which is what a 25-second limit actually constrains [2].

**Compaction and memory.** Compaction summarises older history to free space. It may be automatic, when the window crosses a threshold, or explicit. Long-term memory moves selected facts out of the history into a store, which is read back by a tool or injected into the prompt [12]. Both change the window abruptly, and both can invalidate the cache. Long contexts also hurt answer quality [13, 14, 15]. So the goal is not only a smaller bill but a window that holds what matters.

**Telemetry.** The OpenTelemetry GenAI semantic conventions define spans for agent invocation (`invoke_agent`), model calls (`chat`) and tool execution (`execute_tool`), with attributes such as `gen_ai.usage.input_tokens` and `gen_ai.usage.output_tokens` [5]. MAF emits these spans. MLflow ingests OTLP traces, stores one trace per request and groups traces into sessions through trace metadata [6, 7]. Everything the tape needs, except a breakdown by category, is therefore already collected.

**Evaluation.** Agent quality is measured in three ways: deterministic checks in code (a schema, a word limit, a rule such as "never run a destructive command without asking"), LLM judges that grade an answer against a rubric [27], and signals from the user (a re-asked question, a thumbs-down). The same telemetry stack has a place for each result. MLflow stores *assessments* on a trace, each with a name, a value, a source (code, LLM judge or human) and a rationale, and offers judges over whole conversations [18]. OpenTelemetry defines a `gen_ai.evaluation.result` event with `gen_ai.evaluation.name`, `gen_ai.evaluation.score.value`, `gen_ai.evaluation.score.label` and `gen_ai.evaluation.explanation`, parented to the span that was evaluated [17]. What is missing is a view that puts these results next to the context they were produced from.

# 3 The Context Tape

## 3.1 Record

A session is a sequence of turns $t = 1, \ldots, T$. A turn is one user message and all the work the agent does for it. It maps to one MLflow trace. For each turn we record:

- the end-of-turn context vector $c_t = (c_{t,1}, \ldots, c_{t,K})$ over $K$ categories, and its total $C_t = \sum_k c_{t,k}$;
- the number of model calls $n_t$, and the input of each call $C_{t,j}$, $j = 1..n_t$. The largest of these is the in-turn peak $\hat{C}_t$;
- billed input $I_t = \sum_j C_{t,j}$, of which $I_t^c$ was cached, and billed output $O_t$;
- tool calls by name, memory reads and writes, and compaction events with the context size before and after;
- from version 1.3, the evaluation results attached to the turn (Section 7).

The record holds token counts and scores only. The categories, their colours, and where each sits in the Prompt Stack of [1] are listed in Table 1.

**Table 1:** Context categories. The detailed categories follow Claude Code's `/context` view, with Messages split into user, assistant and tool results, and with a separate band for the compacted summary. The neutral category is what the record stores when a framework does not use Claude Code's names (Section 3.2). The Prompt Stack column gives the concern layer and scope envelope from PIR [1]. The attribute is the span attribute the exporter reads (Section 6).

| Category | Neutral category | Prompt Stack layer / envelope | Span attribute |
|---|---|---|---|
| System prompt | `instructions` | Identity, Policy / global, tenant | `pir.tokens.system_prompt` |
| Skills | `instructions` | Procedures / global | `pir.tokens.skills` |
| System tools | `tool_definitions` | Capabilities / global | `pir.tokens.system_tools` |
| MCP tools | `tool_definitions` | Capabilities / tenant | `pir.tokens.mcp_tools` |
| Memory files | `memory` | Context / session | `pir.tokens.memory` |
| – | `retrieved_evidence` | Context / request | `pir.tokens.retrieved_evidence` |
| Compacted summary | `summary` | Context / session | `pir.tokens.summary` |
| User messages | `conversation` | Context / request (history) | `pir.tokens.user` |
| Assistant messages | `conversation` | Context / request (history) | `pir.tokens.assistant` |
| Tool results | `tool_results` | Context / request (history) | `pir.tokens.tool_results` |
| – | `unattributed` | – | (provider input minus the sum) |

## 3.2 Framework-neutral categories

Claude Code's names are a good default for a coding agent but a poor fit elsewhere: a banking assistant has no MCP tools or skills, and a retrieval agent has a large band of retrieved passages that `/context` has no name for. Version 1.2 therefore stores categories under a neutral set, `instructions`, `tool_definitions`, `memory`, `retrieved_evidence`, `summary`, `conversation`, `tool_results` and `unattributed`, and keeps the detailed names as optional sub-categories that roll up into them (Table 1). The page draws whichever level the data has. In MAF terms, agent instructions map to `instructions`, function tools to `tool_definitions`, context-provider output to `memory` or `retrieved_evidence`, and the chat history to `conversation` and `tool_results`.

`unattributed` closes the books. When the category counts of a call do not add up to the input the provider billed (because the counter used a different tokeniser, or a framework added wrapper tokens), the difference is drawn as its own grey band rather than spread over the others. A thick `unattributed` band is a sign that the counter is in the wrong place, not a property of the session.

## 3.3 Per-call records and multiple agents

A turn-level record assumes one agent and one window. Real deployments often have more: a router on a small model, a main agent on a large one, a sub-agent with its own history, a judge. Each has its own window and its own growth. Version 1.2 adds a per-call record (schema 1.3): one row per model call with `turn`, `agent_id`, `model_id`, `window`, `categories`, `input_tokens`, `cached_input_tokens` and `output_tokens`. The turn-level view is then derived for one agent, named by `primary_agent`, and the page draws a separate per-call tape for each (agent, model) pair, each against its own window line. A sub-agent that compacts on turn 12 while the main agent is at 30% occupancy is visible as such, instead of being averaged away. Volume and cost are still summed over all calls, so the bill matches the provider's.

## 3.4 Visual encoding

The tape has five aligned views (Figure 1). The first four are described here; the fifth, the evaluation tape added in version 1.3, is described in Section 7.

1. **Context histogram.** For each turn, a column stacks $c_t$ in a fixed order. The fixed prefix is at the bottom, then the summary, then the growing history. The columns touch, so the chart reads both as a histogram (one bar per turn) and as an area over time. A step shape is used instead of a smooth area because the context only changes between turns: interpolating would invent states that never existed. A thin tick marks the in-turn peak $\hat{C}_t$ when it is higher than $C_t$, which happens when compaction runs at the end of a turn. Horizontal rules mark the window $W$ and the compaction threshold $\Theta = W - B$. The hatched band above $\Theta$ is the auto-compact buffer $B$.
2. **Event strip.** Ticks count tool calls. A triangle marks a compaction, a diamond a memory write and a circle a memory read. A dashed line runs from each compaction up through the histogram.
3. **Volume panel.** Paired bars show billed input (pale for all input, dark for its uncached part) and output, like trading volume under a price chart. Output is on its own axis because it is two orders of magnitude smaller.
4. **Snapshot grid.** For a selected turn, a $20 \times 10$ grid shows the window as in `/context`. Each square is 0.5% of $W$, the coloured squares come first, and the hatched squares at the end are the buffer (Figure 2).
5. **Evaluation tape.** Under the volume panel, a bar of failing checks per turn stacked by family, and one lane per family (Section 7.3).

Colours follow a categorical palette checked for colour-vision deficiency. The same colours are used in the paper, on the interactive page and in the grid. The page adds a crosshair with a tooltip listing every category for the hovered turn, keyboard stepping, a replay animation and a data table.

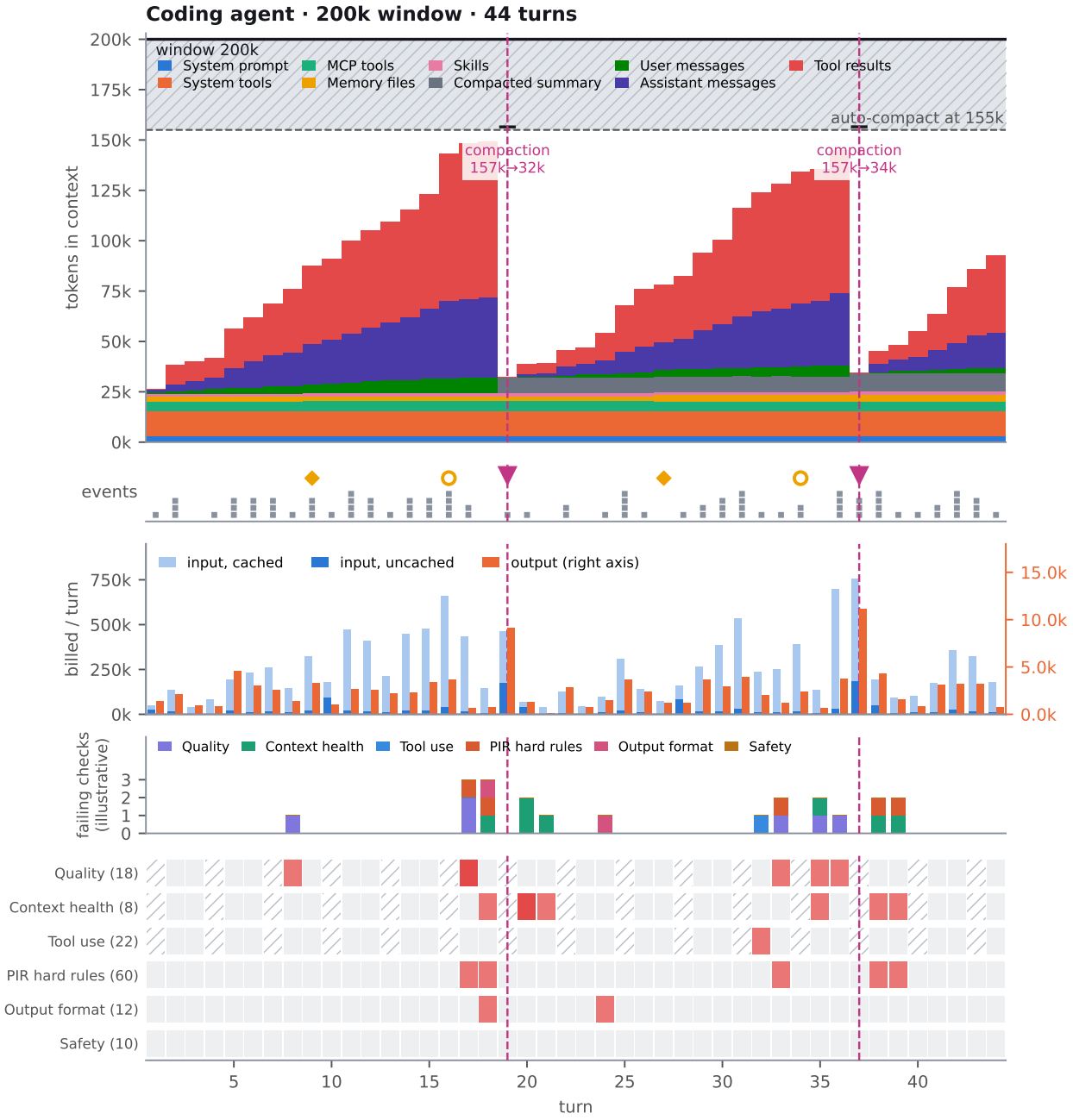



**Figure 1:** The complete Context Tape of the synthetic coding session. From the top: (1) end-of-turn context by category; black ticks mark in-turn peaks above the end-of-turn value, dashed magenta lines mark compactions at turns 19 and 37, and the hatched band is the 45k auto-compact buffer. (2) Events: tool calls (grey ticks), memory writes (⧫), memory reads (○) and compactions (▼). (3) Billed input per turn over all calls (pale: total, dark: uncached), with output on the right axis; uncached spikes sit on the compaction turns and the turns after them, and on the turns after the memory writes (10, 28). (4) Evaluation tape, **illustrative**: failing checks per turn out of 130, stacked by family. (5) One lane per family, shaded by failing checks; hatched cells are turns the sampled LLM judges did not see. Panels 1–3 are the generated session, as in v1.0. The check results in panels 4–5 are *synthetic*, generated from the session state to show the encoding (Section 7.4); they are not measurements. Figure 4 expands two of the families.

# 4 Metrics

The tape makes the following quantities visible. Each one can be computed from the record alone.

**Occupancy** $u_t = C_t/W$. The height of the column against the window line.

**Growth** $g_t = C_t - C_{t-1}$ on turns without compaction. The median and 95th percentile describe how fast a session fills.

**Headroom** $h_t = (\Theta - C_t)/\tilde{g}$, where $\tilde{g}$ is the median growth: an estimate of how many turns remain before compaction. It is the number to alert on.

**Overshoot** $\max_t(\hat{C}_t - \Theta)$. How far a turn goes past the threshold before compaction can run. The buffer $B$ must exceed the largest in-turn growth plus the summary, or a call will overflow the window.

**Call amplification** $a = \sum_t I_t / \sum_t C_t$. How many times, on average, each turn's context is paid for. It is close to the mean number of calls per turn.

**Input-to-output ratio** $\sum_t I_t / \sum_t O_t$.

**Cache retention** $\rho = \sum_t I_t^c / \sum_t I_t$, and its complement, uncached input $\sum_t (I_t - I_t^c)$. A *cache break* is a turn whose cached share falls well below the previous turn's. Each break is attributed to the event just before it (compaction, memory write, prompt change).

**Compaction ratio** $\kappa = C_{\text{after}}/C_{\text{before}}$, and the interval in turns between compactions.

**Category share** $c_{t,k}/C_t$ over time. The tape shows at a glance whether tool results, history or the fixed prefix dominate.

In a multi-agent session (Section 3.3) occupancy, growth, headroom and overshoot are computed per (agent, model) lane against that lane's window, while amplification, ratio and retention are computed over all calls. Evaluation metrics are defined in Section 7.4.

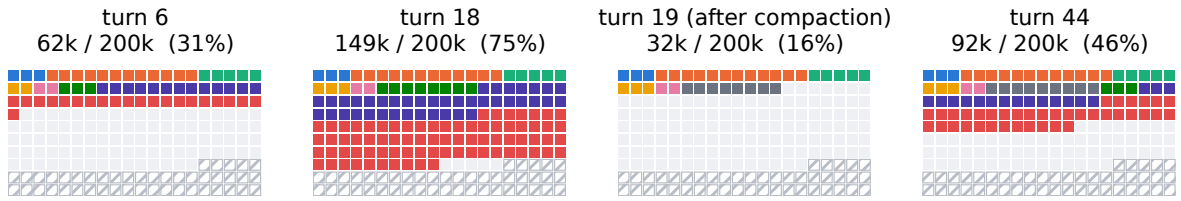



**Figure 2:** `/context`-style snapshots of the coding session. Each square is 0.5% of the 200k window, coloured as in Table 1. Grey squares are free space and hatched squares are the auto-compact buffer. Turn 18 is the last turn before compaction: tool results (red) fill half the window. At turn 19 compaction replaces 157k tokens of history with an 8k summary (grey).

# 5 A Cost Model

Let the fixed prefix be $P$ tokens, let the context grow by $g$ tokens per turn, and let each turn make $k$ calls. To first order the context at turn $t$ is $C_t \approx P + gt$. Billed input over $T$ turns is then

$$\sum_{t=1}^{T} I_t \;\approx\; k \sum_{t=1}^{T} (P + gt) \;=\; k\Big(PT + \tfrac{g}{2}T(T+1)\Big), \tag{1}$$

which is quadratic in $T$. A larger window delays compaction but does not change this growth. It only lets the quadratic term run longer.

With threshold compaction the context follows a sawtooth between the post-compaction level $a$ and the threshold $\Theta$. Its mean is $(\Theta + a)/2$, so the cost per turn is constant and total cost becomes linear in $T$. Let $\gamma$ be the effective price of an input token, mixing cached and uncached tokens (with retention $\rho$ and cached weight $w_c$, $\gamma = \rho w_c + (1-\rho)$). Each compaction costs about

$$K(\Theta) = \Theta + w_o s + a,$$

made up of the compaction call's input $\Theta$, the summary output $s$ at output weight $w_o$, and an uncached refill of about $a$ on the next call. Compactions happen every $(\Theta - a)/g$ turns. The cost per turn is therefore

$$J(\Theta) = k\gamma \frac{\Theta + a}{2} + \frac{g\,K(\Theta)}{\Theta - a}. \tag{2}$$

Setting $dJ/d\Theta = 0$ with $K$ treated as locally constant gives

$$\Theta^\star - a = \sqrt{\frac{2\,g\,K(\Theta^\star)}{k\gamma}}. \tag{3}$$

This is the economic order quantity of inventory theory [16]. Carrying context is the holding cost, paid on every call. Compaction is the ordering cost, paid once per cycle. The best cycle length grows with the square root of the growth rate and the compaction cost, and shrinks with the square root of the number of calls per turn and the input price. Equation 3 is solved by fixed-point iteration.

Two cautions apply. First, the model prices tokens and nothing else. Each compaction loses detail, and a lower threshold means more summaries of summaries. $\Theta^\star$ is a lower bound on how aggressive compaction should be, not a recommendation to compact as often as it says. The evaluation tape (Section 7) is where the other side of this trade-off becomes visible. Second, the threshold also has a hard floor set by overshoot: $W - \Theta$ must exceed the largest in-turn growth plus $s$.

# 6 Instrumentation with OpenTelemetry and MLflow

Figure 3 shows the data path. Three spans are already emitted by MAF and other OpenTelemetry-instrumented frameworks. Two things are added for the context tape, and a third for the evaluation tape (Section 7.2).

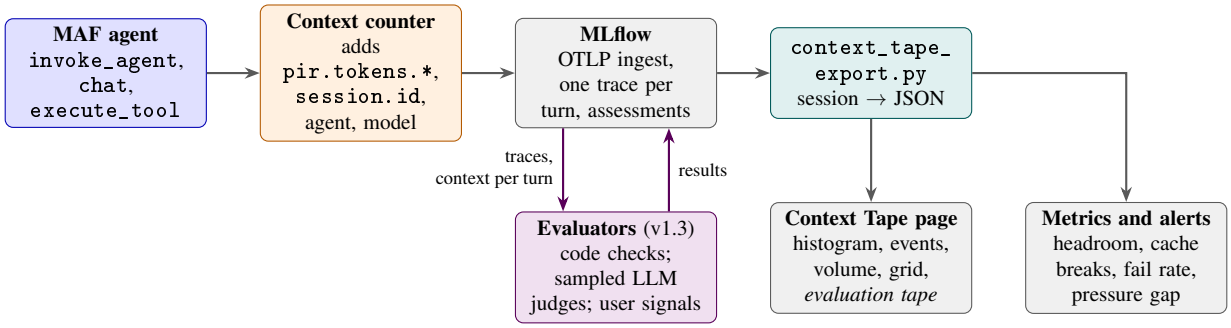



**Figure 3:** Data path. Only token counts and evaluation results leave the service. The context counter is the one new component on the request path: it counts tokens per category when the request is assembled and attaches the counts to the model-call span. Evaluators (new in v1.3) read traces back, judge each turn against the context the model saw, and write results to the same traces (`gen_ai.evaluation.result` events or MLflow assessments), so the exporter can join them by trace and turn.

**Session identity.** Each turn's trace carries the session identifier, either as MLflow trace metadata (`mlflow.trace.session`, set with `mlflow.update_current_trace`) or as a `session.id` attribute on the root span [7]. The exporter accepts both.

**Category counts.** When the request is assembled, a small counter tokenises each part and attaches `pir.tokens.{category}` to the chat span. On a compaction it also attaches `pir.compaction.before` and `pir.compaction.after`. The parts are already separate at assembly time, so this costs one tokeniser pass and needs no prompt text in telemetry. In MAF the natural place is a chat-client middleware, which sees the final message list before it is sent. The same middleware records the agent and model of the call (`gen_ai.agent.name`, `gen_ai.request.model`), which is all the per-call record of Section 3.3 needs. Without category attributes the tape still works. Each turn becomes a single `unattributed` band equal to the last call's input, and every metric except category share is still available.

**Algorithm 1** Exporting a session from MLflow to the tape record

$\mathcal{T} \leftarrow$ SEARCHTRACES(experiment, session $= s$), ordered by start time
**for** each trace $\tau_t \in \mathcal{T}$ **do** ▷ one trace = one turn
  $I_t, I_t^c, O_t, n_t, \hat{C}_t \leftarrow 0$; tools $\leftarrow \{\}$; events $\leftarrow []$
  **for** each span $\sigma$ in $\tau_t$ in start order **do**
    **if** $\sigma$ is a model call **then**
      add input, cached and output tokens; $n_t \mathrel{+}= 1$; $\hat{C}_t \leftarrow \max(\hat{C}_t, \text{input})$; last $\leftarrow \sigma$
      append a per-call row (agent, model, window, `pir.tokens.*`, tokens) ▷ schema 1.3
    **else if** $\sigma$ is a tool call **then**
      count tool name; add a memory event if the name is a memory tool
    **end if**
    **if** $\sigma$ is a compaction **then**
      record before/after; add a compaction event
    **end if**
  **end for**
  $c_t \leftarrow$ `pir.tokens.*` of last, plus `unattributed` $=$ input of last $-\sum$ categories
  evals$_t \leftarrow$ assessments and `gen_ai.evaluation.result` events on $\tau_t$ ▷ schema 1.4, Section 7.2
**end for**
**return** {session, $W$, $B$, primary_agent, turns, calls}

**Export.** Algorithm 1 turns one session into the record. Model calls are recognised by `gen_ai.operation.name = chat` or by MLflow's chat span type. Cached tokens come from `gen_ai.usage.cache_read.input_tokens` where the provider reports them. Tools whose name contains `memory` are classed as memory reads or writes. On a compaction turn the last model call saw the context before compaction, so the exporter keeps the fixed-prefix categories and sets the summary to the remainder of `pir.compaction.after`.

# 7 The Evaluation Tape

The panels above say what the model was given and what it cost. They do not say whether the answers were any good. Quality and context are linked: answers degrade as the window fills [13, 14, 15], a compaction can drop a fact the user gave twenty turns earlier, and a cost policy such as capping tool results is only worth adopting if it does not make answers worse. The evaluation tape puts per-turn quality results directly under the volume panel, on the same turn axis, so that each failure can be read against the context that produced it.

## 7.1 Sources

Table 2 lists the three sources. They differ in cost, coverage and trust, and the tape draws them differently for that reason.

**Table 2:** Sources of per-turn evaluation results.

| Source | Examples | When and at what cost | How it is shown |
|---|---|---|---|
| Code checks | PIR hard rules turned into assertions ("ask before a destructive command", "cite changed file paths"); output schema; word limit; diff parses | Inline or just after the turn; every turn; near-free | Pass or fail per check; always present |
| LLM judges | Groundedness in tool output; task completion; knowledge retention; tool-argument validity; instruction adherence [27, 18] | Offline, on a sample of turns plus every turn a code check flagged; costs a model call per judge | Score in $[0, 1]$ with a threshold; hatched where not run |
| User signals | Re-asked question, correction, thumbs-down, abandonment | When the user acts, often one turn later | Mark on the turn the signal is about, not the turn it arrived on |
| Session-level | Conversation completeness, user frustration [18] | Once per session | Badge at the end of the tape |

PIR [1] is a natural source of code checks. Its parser extracts the hard rules of a prompt (constraints, prohibitions, output formats) as tagged nodes. Each one that can be checked mechanically becomes a per-turn assertion. A long prompt yields dozens of these, which is one reason the tape must handle many checks.

## 7.2 Collection

**Join keys.** Every result carries the trace id of the turn and, where it is about one call, the span id or `gen_ai.response.id` of that call. In MLflow a result is logged as an assessment with `mlflow.log_feedback(trace_id, name, value, source, rationale)`; session-level results attach to the first trace of the session [18]. In OpenTelemetry it is a `gen_ai.evaluation.result` event parented to the evaluated span, with `gen_ai.evaluation.name` (required), `score.value`, `score.label` and `explanation`, and `error.type` when the evaluator itself failed [17]. The exporter reads both and writes them into the turn record (Algorithm 1, line 16).

**Judge what the model saw.** A judge that grades turn 30 must see the context the model had at turn 30, not the full transcript. After a compaction that context is the summary plus the turns since, and a fact the summary dropped is exactly what a knowledge-retention judge is meant to catch. The tape record says which categories were present and when compactions happened, so the evaluator can rebuild the per-turn input from the trace (or, where traces hold no text, call the agent's own context assembler on the stored history).

**Keep judge tokens off the volume panel.** Judge calls are billed, but they are not the agent's cost. They are recorded under a separate `agent_id` (for example `judge`) and excluded from the volume panel and from Tables 6–8, with their own total shown beside the evaluation tape.

**Make scores comparable.** Each check stores its value normalised to $[0, 1]$ with higher meaning better, its threshold, and a label (`pass`, `warn`, `fail`). LLM judges also store the judge model and prompt version, because a changed judge changes the score with no change in the agent [27]. A result whose evaluator errored or did not run is stored as missing, never as a pass.

**Store sparsely, pre-aggregate.** Most checks pass on most turns. The record therefore stores every non-pass result in full, and for each turn and family only an aggregate: number of checks run, number failing, worst score and the names of failing checks (schema 1.4, Appendix A). The page draws the summary and family views from aggregates alone and fetches full results only when a family is expanded.

## 7.3 Visual encoding at scale

One row per check works for five checks. At a hundred checks and two hundred turns it is a 20,000-cell grid in which the few failures that matter are lost. The evaluation tape is therefore drawn in three levels: the first two sit under the volume panel in Figure 1, and the third opens on demand (Figure 4).

1. **Summary bar.** One bar per turn: the number of failing checks, stacked by family colour. It is the only evaluation panel always shown, and at a glance it says where problems cluster.
2. **Family lanes.** Six to eight lanes, one per family, each cell shaded by how many checks in the family failed on that turn. Families are groups the reader already thinks in: PIR structure (hard rules, output format), judge type (quality, context health, tool use), or risk (safety). Passing cells are a quiet grey; failures are red; turns a sampled judge did not see are hatched, so missing data never reads as a pass.
3. **Failures-only drill-down.** Expanding a family lists only the checks that failed at least once in the session, ordered by number of failures, with a final line "+$N$ checks passed on every turn". Sixty PIR hard rules become four rows.

Compaction lines continue through all three levels, so a failure just after a compaction lines up with the dashed line above it. The page offers three orderings for the drill-down: *most informative* (most failures first), *regressions* (checks that fail now but passed on a baseline run of the same workload), and *search and pin*, which keeps chosen checks visible regardless of result. Above roughly 5,000 cells the page draws the lanes on a canvas rather than as SVG elements.

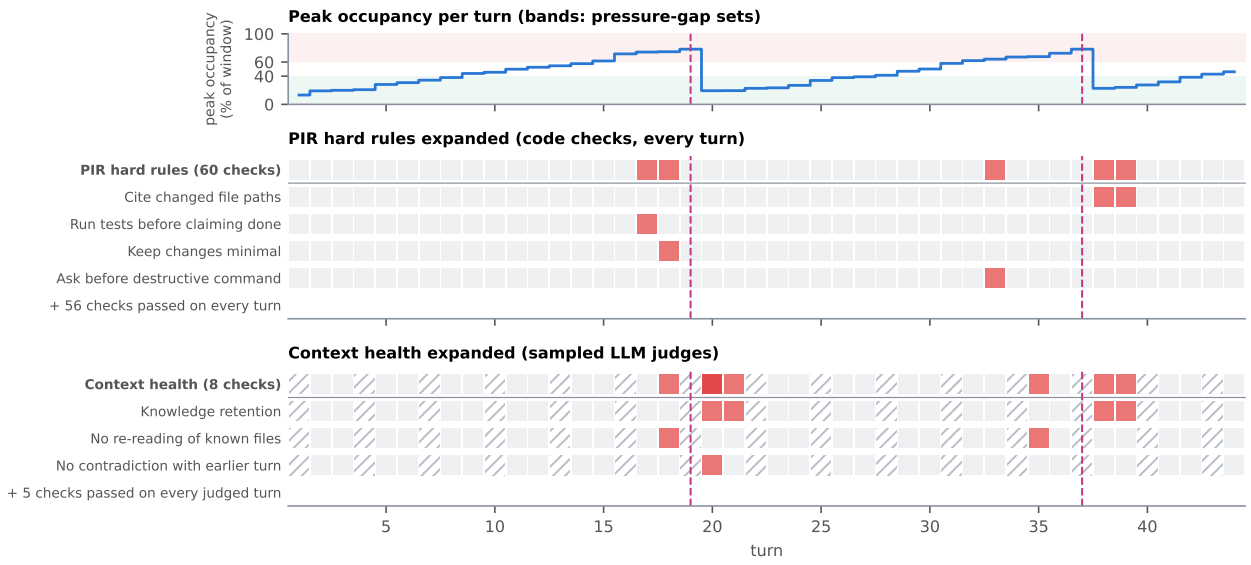



**Figure 4:** Drill-down of the evaluation tape in Figure 1, **illustrative**. Top: in-turn peak occupancy from the record, with the low ($< 40\%$) and high ($> 60\%$) bands used for the pressure gap. Middle: the PIR hard-rules family (60 code checks) expanded; only the four checks that ever failed get a row, and 56 are folded into one line. Bottom: the context-health family (8 LLM-judge checks) expanded; hatched cells are turns the sampled judges did not see, so they read as unknown, not as passes. Check results are synthetic, as in Figure 1. Across all six families, 5,720 check–turn results reduce to one bar, six lanes and, when opened, a few rows per family.

### 7.4 Evaluation metrics

**Fail rate per family** $F_f = \frac{1}{|T_f|} \sum_{t \in T_f} \mathbb{1}[\text{some check in } f \text{ fails at } t]$, over the turns $T_f$ on which family $f$ ran.

**First failing turn** $t_f^0$, the first turn on which family $f$ fails. Compared with the first compaction turn, it says whether trouble starts before or after the window is rewritten.

**Pressure gap** The mean score (or pass rate) on turns with $u_t < u_{\text{lo}}$ minus that on turns with $u_t > u_{\text{hi}}$, using the in-turn peak $\hat{C}_t / W$. A positive gap means quality falls as the window fills. We use $u_{\text{lo}} = 0.4$ and $u_{\text{hi}} = 0.6$ or $0.7$ depending on the window.

**Post-compaction dip** The pass rate on the $m$ turns after each compaction (we use $m = 2$) against all other turns. It prices the quality side of Equation 3: if lowering $\Theta$ raises the number of compactions, the dip is paid more often.

**Judge coverage** The share of turns on which each judge ran. A fail rate computed at 20% coverage is reported with that coverage beside it.

On the illustrative data of Figures 1 and 4, 12 of 44 turns have a failure. Turns whose in-turn peak exceeds 60% of the window fail on 6 of 11 (55%) against 6 of 21 (29%) below 40%; the four turns just after the two compactions all fail, against 20% of other turns; the judges cover 29 of 44 turns (66%). These numbers restate how the illustration was generated and say nothing about real agents. They show that the metrics can be read straight off the record.

## 8 Comparing Prompt Versions

The evaluation tape says whether one session went well. A prompt change needs a different answer: is the new version better than the current one on the same conversations? A single run cannot say this. Models are stochastic, judges are noisy, and in a multi-turn session a small change early on alters every turn after it, including when the window compacts. This section turns the tape into a repeatable comparison and a loop: a fixed multi-turn suite, a rubric, a paired comparison with a decision rule, a diff tape for reading what changed, and a champion–challenger loop that keeps the best version so far (Algorithm 2).

## 8.1 A fixed multi-turn suite

A *scenario* is a multi-turn test that can be run against any prompt version:

- a goal and a persona, and either scripted user turns or a user simulator whose model, prompt and temperature are pinned (MLflow's conversation simulator takes a goal, persona and guidelines per test case; it is experimental in MLflow 3.10 [19]);
- *tool fixtures*: recorded tool responses keyed by tool name and arguments, so that every version sees the same world. A call with no fixture returns a fixed "not available" result and is flagged, because it means the version behaved differently enough to leave the recorded path;
- planted facts with *probe* turns placed after the expected compaction points, so that knowledge retention is exercised on every run rather than when it happens to come up;
- expectations for session-level judges, such as the outcome the user should end with.

Scenarios are split into a *dev* set, whose failures may be read by whoever writes the next candidate, and a *held-out* set, used only when a candidate is up for promotion. Sessions that fail in production, found on real tapes, become new dev scenarios. Every scenario is run $k$ times per version (we use $k = 5$).

Each run is an ordinary tape record with a `run` block (schema 1.5, Appendix A): prompt name and version, the parent version and a one-line change, the suite version, scenario, split, repeat, mode, agent model and the pinned judge. Next to its version label the run stores a content hash of the rendered prompt, so two labels that render the same text are recognisably one prompt. Because the suite is small, every judge runs on every turn: coverage is 100% and no failure hides in an unjudged turn.

## 8.2 Pairing: replay and live

There are two ways to run a candidate on a scenario, and they answer different questions (Table 3).

**Table 3:** Two ways to run a candidate prompt on the same scenario.

| | Replay | Live |
|---|---|---|
| How | The candidate answers each turn of the champion's recorded session: the history is the champion's, only the prompt is swapped | The candidate runs the whole scenario against the scripted user (or simulator) and tool fixtures |
| Paired by | Turn: both versions answer the same context | Scenario: trajectories diverge after the first different answer |
| Captures | The prompt's effect on single responses | Compounding, compaction timing, cost, session-level outcomes |
| Misses | The candidate never sees its own earlier answers; compactions stay where the champion had them | Nothing by design, but needs repeats to separate the prompt's effect from sampling noise |
| Use | Screening many candidates cheaply | Deciding |

A change to the prompt also changes the fixed prefix, and with it the cost and the compaction turns. In Figure 5 a candidate 400 tokens longer compacts one turn earlier than the champion. Replay cannot show this; live runs can, and the tape makes it visible.

## 8.3 A rubric of prompt quality

The rubric has three layers, applied in order.

1. **Gates.** Any one blocks promotion, whatever the score. (i) Zero tolerance: any failure in a zero-tolerance family (safety). (ii) No regression in gated families (PIR hard rules): no check may fail on more turns per run than the champion by more than the largest change any check shows in an A/A run (below). (iii) Cost budget: billed input at most $1.05\times$ the champion's.
2. **Quality score.** For run $r$ of version $v$ on scenario $s$,

$$Q(v,s,r) = \frac{\sum_f w_f P_f}{\sum_f w_f}, \qquad P_f = 1 - F_f, \tag{4}$$

where $F_f$ is the fail rate of family $f$ from Section 7.4: the share of judged turns on which some check in $f$ fails. Session-level judges enter as a family with a single turn. The weights are a product decision and are written in a rubric file next to the suite; we use quality .25, context health .20, tool use .15, PIR hard rules .25 and output format .15.

3. **Diagnostics.** Reported for every comparison but not in $Q$: billed and uncached input, compactions per run, pressure gap, post-compaction dip, and per-check changes in failing turns per run. Consistency over repeats can be added as pass$^k$, the share of scenarios on which all $k$ repeats meet a target [22].

The rubric is versioned like the prompt. A change of weights, checks or judge makes earlier scores incomparable, so the champion is re-run under the new rubric before the next comparison.

## 8.4 Deciding

The scenario is the unit of comparison; repeats are averaged inside it. For champion $b$ and candidate $c$,

$$\Delta_s = \bar{Q}(c,s) - \bar{Q}(b,s), \qquad \Delta Q = \frac{1}{|S|}\sum_{s\in S}\Delta_s, \tag{5}$$

with a 95% interval $[\Delta_{\text{lo}}, \Delta_{\text{hi}}]$ from a bootstrap over scenarios. Before the first comparison the champion is run again with fresh repeats (an *A/A run*). Its interval should cover zero; its half-width is the measurement noise, and the non-inferiority margin $\delta$, the largest quality loss accepted in exchange for a saving, must exceed it, or the suite needs more scenarios or repeats. We use $\delta = 0.015$. A candidate is then:

- **promoted as better** if it passes the gates and $\Delta_{\text{lo}} > 0$;
- **promoted as cheaper, no worse** if it passes the gates, its billed input is at least 2% lower and $\Delta_{\text{lo}} > -\delta$;
- in either case only if the held-out split does not contradict it: $\Delta Q_{\text{held}} \geq -\delta$ and $\Delta_{\text{hi,held}} \geq 0$;
- **inconclusive** if its interval straddles the boundary it needs to clear. It gets one extension, declared in advance (to $k = 10$ repeats on dev), and is then decided. Further looks would raise the rate of false promotions;
- **rejected** otherwise.

When several candidates are tried in one round, the best of them is biased upward by the selection itself, so the winner is confirmed on fresh repeats before it is promoted.

## 8.5 The diff tape

Figure 5 draws two versions on one scenario. The top panel overlays their occupancy, so a shift in compaction turns is visible. The butterfly panel puts the champion's failing families above the axis and the candidate's below, summed over repeats, on the same turn axis as everything else. The delta lanes show, per family and turn, how many more or fewer repeats failed with the candidate; the totals on the right are the per-family change. Expanding a lane lists the checks that changed, ordered by change in failing turns per run. This gives the *regressions* ordering of Section 7.3 a concrete baseline: the champion on the same scenario.

## 8.6 The loop

Algorithm 2 closes the loop. Candidates come from three sources: an engineer's edit, a PIR refactor [1] (which should leave behaviour unchanged and lower cost), or a prompt optimiser given the champion's failing checks and their explanations, as in OPRO, DSPy, TextGrad and GEPA [23, 24, 25, 26]; MLflow wraps GEPA as `mlflow.genai.optimize_prompts` [21]. Each candidate makes one targeted change, recorded as a PIR diff against its parent, so a promotion or rejection can be traced to what changed. Promotion moves an alias in the prompt registry (`mlflow.genai.set_prompt_alias(name, alias="champion", version=n)`), and the agent loads `prompts:/name@champion` [20], so rolling back is moving the alias back.

The loop overfits if it is allowed to. The guards are the held-out split, which no candidate writer or optimiser sees; one targeted change per candidate; a pinned judge, because a better score from a changed judge is not a better prompt [27]; and a versioned rubric. An optimiser is the most likely source of overfitting, since it is built to raise the dev score.

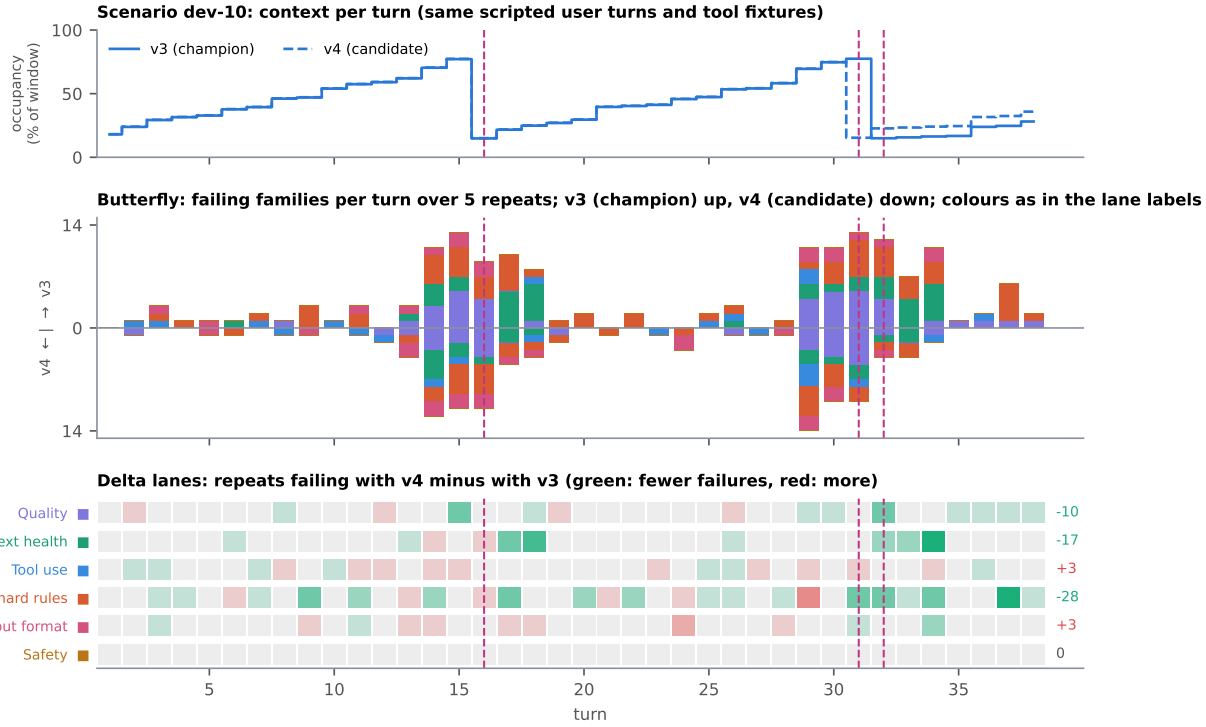



**Figure 5:** Diff tape of champion v3 and candidate v4 on dev scenario 10, **illustrative**, five repeats each. Top: occupancy; both compact at turn 16, then v4, 400 prefix tokens longer, compacts at turn 31 and v3 at 32 (dashed lines mark both). Middle: failing families per turn, v3 above the axis, v4 below. Bottom: delta lanes, repeats failing with v4 minus repeats failing with v3; green cells are turns v4 fixed, red cells turns it broke. Most of v4's gain sits after the compactions, in context health and PIR hard rules, which is what its change targeted. Check outcomes are synthetic (Section 8.7).

**Algorithm 2** Champion–challenger loop over prompt versions

$b \leftarrow$ current prompt; run suite (dev and held-out, $k$ repeats, live) ▷ tapes with `run` blocks
run $b$ again with fresh repeats (A/A); check its interval covers 0 and its half-width $< \delta$
**repeat**
  read $b$'s dev tapes: failing checks, turns, explanations, columns above them ▷ Sections 7.3, 8.5
  propose candidates $c_1..c_m$ (edit, PIR refactor, optimiser on dev failures only)
  optionally screen them in replay mode; keep the best few
  **for** each candidate $c$ **do**
    run $c$ live on dev and held-out; compute $\Delta Q$, interval, gates, cost ▷ Eqs. 4, 5
    decide (Section 8.4); if inconclusive, extend once and decide
  **end for**
  **if** some $c$ is promoted **then**
    confirm the best on fresh repeats; $b \leftarrow c$; move the registry alias
  **end if**
  add scenarios from production failures to dev; occasionally refresh held-out
**until** no promotion in $R$ rounds, or budget spent

## 8.7 Illustration

We ran the loop on a synthetic suite to exercise each decision path. The suite has 16 dev and 8 held-out scenarios from the coding-session generator of Section 9, each with its own seed, 30–44 turns and a difficulty factor. There are six versions, including an A/A re-run of v1, and 880 runs in total. Context and tokens come from the generator, with the prefix sized per version, so cost and compaction differences are produced by the generator. Check outcomes are *synthetic*: each check fails with a probability set by the session state (pressure, the two turns after a compaction,

large tool results), scaled by the scenario's difficulty and by per-version multipliers that we set by hand, inherited from parent to child. Every number below restates how the illustration was generated, run through `compare.py`; none is a measurement of a real prompt. Figure 6 and Table 4 show the result.

**Table 4:** Rounds of the loop on the illustrative suite. $\Delta Q$ on dev with the 95% bootstrap interval over 16 scenarios; held-out over 8. Billed is candidate over champion. Outcomes are synthetic.

| Cand. | vs | Prefix | Change | $\Delta Q$ dev | $\Delta Q$ held | Billed | Decision |
|---|---|---|---|---|---|---|---|
| v1 | v1 | 24.1k | A/A: same prompt, fresh repeats | $-0.001$ [$-0.009$, $+0.007$] | – | 1.000 | noise |
| v2 | v1 | 26.6k | Rule and two examples for citing changed paths | $-0.008$ [$-0.016$, $+0.000$] | $-0.010$ | 1.011 | reject (gate) |
| v3 | v1 | 19.6k | PIR refactor of prompt, skills and tool text | $-0.001$ [$-0.012$, $+0.009$]* | $+0.008$ | 0.979 | promote (cheaper) |
| v4 | v3 | 20.0k | Carry user facts and open tasks through compaction | $+0.021$ [$+0.014$, $+0.030$] | $-0.003$ | 1.000 | promote (better) |
| v5 | v4 | 21.6k | Optimiser rewrite from v4's dev failures | $+0.029$ [$+0.022$, $+0.036$] | $-0.028$ | 1.006 | reject (held-out) |

*After one extension to 10 repeats; the first look gave $-0.003$ [$-0.016$, $+0.009$], which straddles $-\delta = -0.015$.

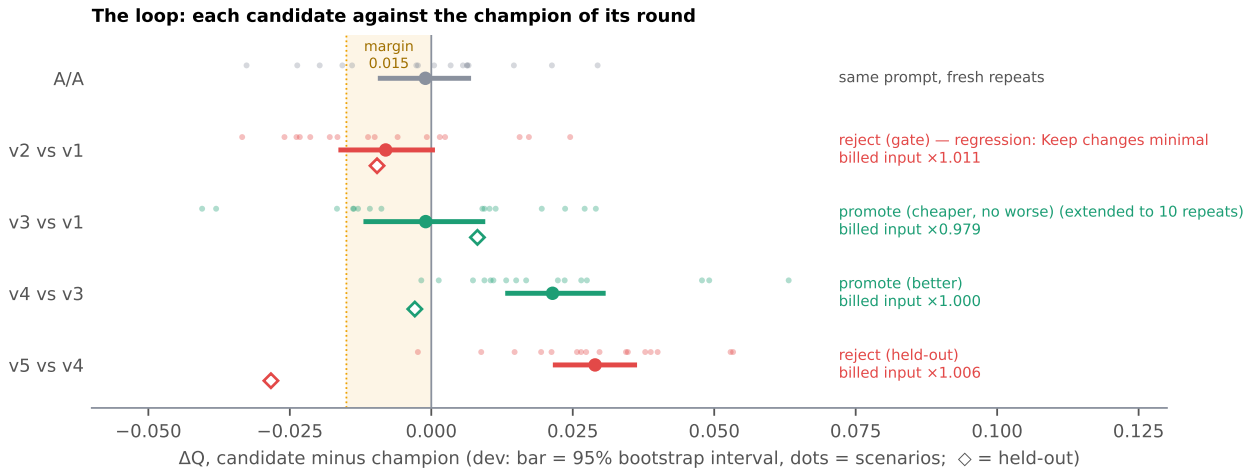



**Figure 6:** The loop on the illustrative suite. Each row compares a candidate with the champion of its round: the bar is the 95% bootstrap interval of $\Delta Q$ on dev, the dot its mean, the faint dots the 16 per-scenario differences and the diamond the held-out mean. The shaded band is the non-inferiority margin. v3 is promoted as cheaper and no worse, v4 as better; v2 fails a gate and v5 improves dev while losing on held-out. Outcomes are synthetic.

Each round shows one part of the rubric at work. **v2** fixes the rule it targets: "cite changed file paths" fails on 2.6 fewer turns per run. But its examples make replies longer ("reply under 300 words" +4.3 turns per run) and edits broader ("keep changes minimal" +0.9 per run, against an A/A maximum of 0.45 for any check). The hard-rule gate rejects it whatever its score, which is also slightly lower. **v3**, a PIR refactor removing 4,500 prefix tokens, lowers billed input by 2.1%, uncached input by 4.2% and compactions from 2.00 to 1.88 per run. Its first interval straddles the margin; after the extension it clears it and v3 is promoted on cost. Its interval is wider than the A/A interval even at 10 repeats, because a shorter prefix moves the compaction turns and so weakens the pairing between versions, which is why live comparisons need repeats. **v4** targets compaction. Knowledge-retention failures fall by 2.0 turns per run and contradictions by 0.9, almost all just after compactions (Figure 5). The held-out estimate ($-0.003$, interval $[-0.017, +0.010]$) does not confirm the gain but does not contradict it. The held-out split is a guard, not a second test. **v5** is the clearest dev result in the table, and the held-out split shows it to be overfitting: $-0.028$ with an interval entirely below zero. The final champion is v4. It is 4,100 prefix tokens smaller than v1 and, compared with v1 directly, $+0.018$ in $Q$ on dev (interval $[+0.006, +0.031]$).

# 9 Evaluation

This section evaluates the cost side of the tape. The evaluation tape of Section 7 has not yet been run on real sessions (Section 11).

## 9.1 Setup

We have not yet exported production sessions, so we evaluate on two synthetic sessions whose parameters are set to plausible values (Table 5). A session is drawn once as a fixed workload: user message sizes, calls per turn, tool choices, tool result sizes and output sizes. Each policy then replays the same workload, so differences between policies are due to the policy alone. A call's cached input is 97% of the previous call's input when the prefix is unchanged, and zero after a compaction or a memory write placed in the prefix. Compaction runs at the end of a turn when $C_t > \Theta$. Its call reads the whole context, writes the summary, and in this model does not reuse the cache, which is the worst case. Cost is in units of one uncached input token, with cached input at $w_c = 0.1$ [8] and output at $w_o = 4$. The output weight is an assumption: it changes the totals but not the conclusions, because output is under 1% of tokens. The same generator, with the same seeds, drives the interactive page, so its numbers match the tables.

**Table 5:** Synthetic session parameters. Ranges are uniform. "Calls" is the maximum number of model calls per turn; every call but the last makes one tool call.

| | Coding agent | Banking assistant |
|---|---|---|
| Window $W$ / buffer $B$ | 200k / 45k ($\Theta = 155$k) | 128k / 25.6k ($\Theta = 102.4$k) |
| Turns | 44 | 70 |
| Fixed prefix | 24.1k: system 3.1k, tools 12.4k, MCP 4.8k, memory 2.2k, skills 1.6k | 7.3k: system 2.6k, tools 3.8k, memory 0.9k |
| User / output per call | 60–700 / 150–1,300 | 30–260 / 90–500 |
| Calls per turn | 1–5 | 1–4 |
| Tool results | Read 0.9–9k, Grep 0.2–1.8k, Bash 0.15–2.6k, Edit 0.1–0.7k, WebFetch 1.5–6k | account 0.4–2.5k, transfers 0.25–1.2k, transactions 0.9–5.2k, card 0.15–0.6k, KB 0.6–3k |
| Memory writes / reads | turns 9, 27 / 16, 34 | turns 6, 18, 33, 47, 61 / 12, 25, 40, 55, 66 |
| Summary size | 7–11k | 2.5–4k |

**Table 6:** Baseline metrics of the two sessions. Cost is in uncached-input-token units ($w_c = 0.1$, $w_o = 4$).

| Metric | Coding | Banking |
|---|---|---|
| Model calls (per turn) | 132 (3.0) | 186 (2.7) |
| Median / p95 / max growth per turn | 6.7k / 14.8k / 20.1k | 2.8k / 7.5k / 10.4k |
| Largest in-turn growth | 19.0k | 9.9k |
| Peak context, end of turn or in turn | 156.5k (78%) | 106.3k (83%) |
| Overshoot past $\Theta$ | 1.5k | 3.9k |
| Compactions (turns) | 2 (19, 37) | 2 (34, 63) |
| Compaction ratio $\kappa$ | 0.20, 0.22 | 0.11, 0.12 |
| Peak share of tool results | 52% | 65% |
| Billed input / output | 11.54M / 113k | 10.18M / 61k |
| Input-to-output ratio | 102 : 1 | 168 : 1 |
| Call amplification $a$ | 3.19 | 2.79 |
| Cache retention $\rho$ | 89.6% | 89.4% |
| Uncached input | 1.20M | 1.07M |
| Compaction calls, share of billed input | 2.7% | 2.1% |
| Cost (M units) | 2.69 | 2.23 |

## 9.2 What the tape shows

Table 6 gives the baseline metrics. Figure 1 and Figure 7 show the tapes. Five observations stand out.

1. **Input dominates by two orders of magnitude.** The model reads 102 to 168 tokens for every token it writes. Each turn's context is paid for about three times, once per call. Shortening answers barely matters. Shrinking what is resent does.
2. **Tool results are the largest growing category.** Just before each compaction, tool results take 52% (coding) and 65% (banking) of the window. Assistant and user messages together take less. This is the red band in both tapes.
3. **Retention is high, but the misses are concentrated.** About 90% of input is cached, yet the uncached 10% carries nearly half of the cost (1.20M of 2.69M units in the coding session) and all of the prefill time. Most uncached tokens fall on a handful of turns: the first turn, the compaction turns and the turns after them, and the turns after memory writes (Figure 1, volume panel). On those turns the cached share drops from about 95% to 45–75%.
4. **The threshold is crossed inside a turn.** Compaction runs between turns, so a tool loop can carry the context past $\Theta$. Here the overshoot is 1.5k and 3.9k tokens. The largest in-turn growth, 19k for coding, is the real lower bound on the buffer.
5. **The fixed prefix sets the floor.** After compaction the coding session restarts at 32–34k, of which 24.1k is the fixed prefix. The banking session restarts at 12–13k. A bloated system prompt or tool list raises every tooth of the sawtooth. This is the cost that PIR attacks.

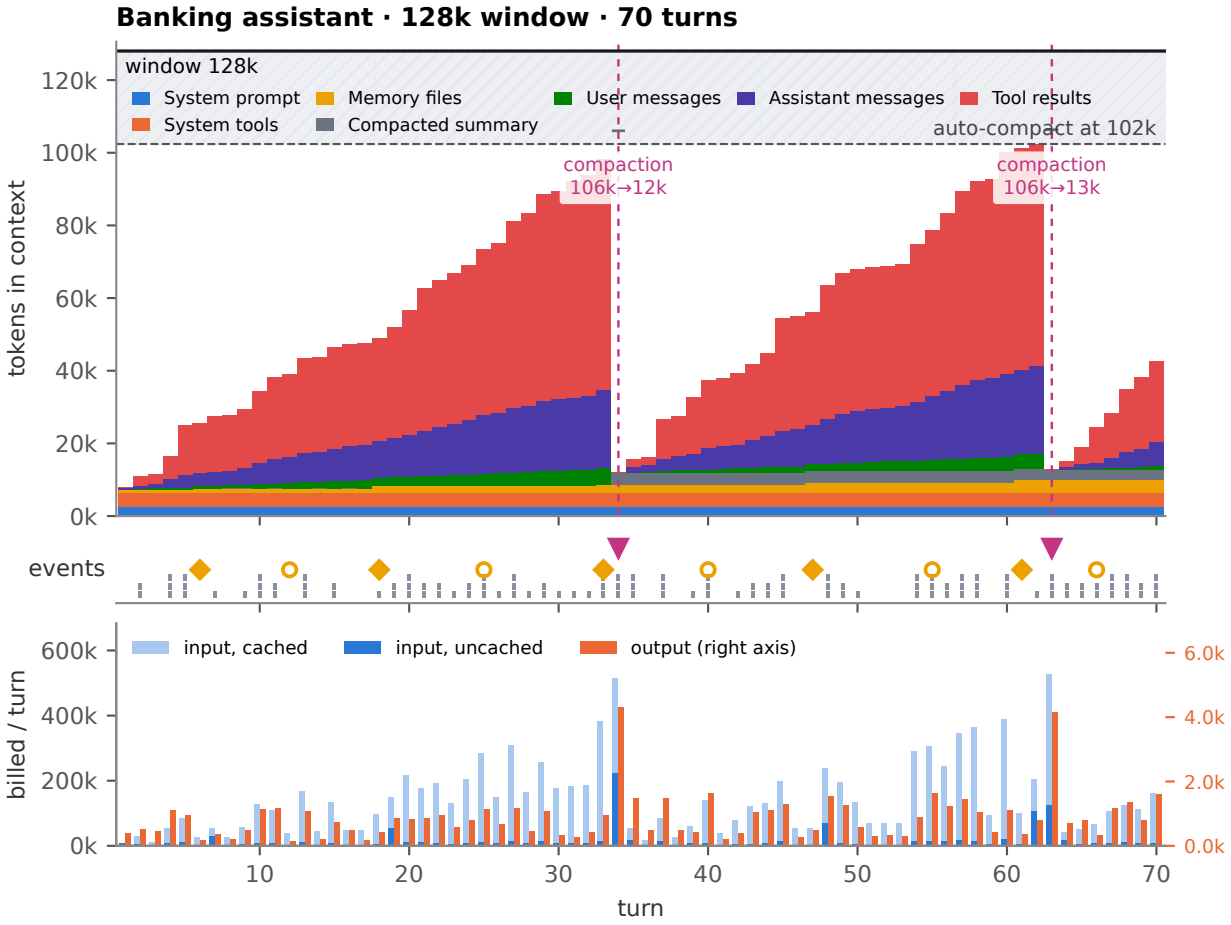



**Figure 7:** Context Tape of the synthetic banking session. The fixed prefix is small (7.3k), so the history dominates and compaction removes almost 90% of the window. Each of the five memory writes (⧫) changes the prefix and is followed by an uncached turn.

## 9.3 Ablations

Table 7 replays each workload under three changed policies. Figure 8 shows cumulative cost.

**A bigger window is not free.** With a 1M window and no compaction, the coding session peaks at 340k and billed input doubles (11.5M to 22.9M). Retention rises to 94% because nothing breaks the prefix, so the cost rises by a smaller 43%. The mean context the model must attend to also doubles (82k to 179k), and long-context quality loss [13, 14] is not priced in here.

**Where memory goes matters more than how big it is.** The memory files add only 1–3k tokens. But when a write changes the system message, everything after it is uncached on the next call. Appending new memories as a message at the end of the history, and folding them into the prefix only at the next compaction (which breaks the cache anyway), removes those breaks. That cuts uncached input by 13% and 29%, with no change in billed input. The banking session gains more because it writes memory five times. This is the Prompt Stack rule from [1] applied over time: session-scoped content must not sit above global content in the prefix.

**Capping tool results delays the first compaction.** Truncating each result (keeping a handle so the agent can read more) moves the first compaction from turn 19 to 26 in the coding session and from 34 to 45 in banking, and removes one compaction in each. It gives the largest cost reduction of the three, 12% and 18%. Whether answers suffer depends on the tool. A truncated file read can usually be reread in parts, while a truncated transaction list may hide the line the user asked about. This is the kind of question the evaluation tape is for: run the same evaluations on the baseline and the capped replay and compare family fail rates in the *regressions* view.

**Table 7:** Policy ablations on the same workloads. Changes are relative to the baseline policy.

| Session | Policy | Compactions | Uncached input | Billed input | Cost |
|---|---|---|---|---|---|
| Coding | Baseline (Table 6) | 2 | 1.20M | 11.54M | 2.69M |
| | No compaction, 1M window | 0 | +7.9% | +98.8% | +43.1% |
| | Memory appended, not in prefix | 2 | −13.1% | 0 | −5.3% |
| | Tool results capped at 2k | 1 | −27.0% | −0.4% | −12.0% |
| Banking | Baseline | 2 | 1.07M | 10.18M | 2.23M |
| | No compaction, 1M window | 0 | +27.1% | +110.7% | +61.2% |
| | Memory appended, not in prefix | 2 | −29.4% | 0 | −12.8% |
| | Tool results capped at 1k | 1 | −31.1% | −8.1% | −17.8% |

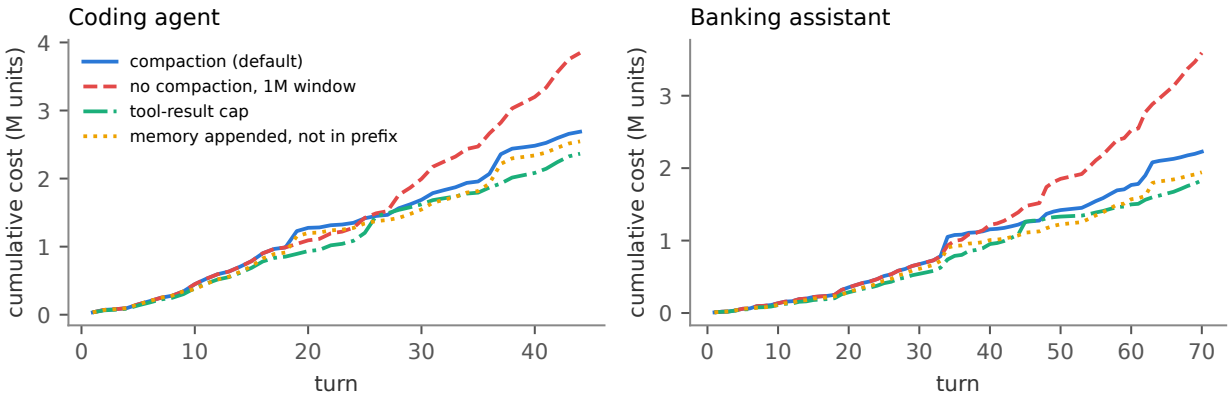



**Figure 8:** Cumulative cost over turns. Without compaction (red, dashed) the curve bends upward, as Equation 1 predicts. With compaction it is close to linear, with steps at each compaction. Capping tool results (green) and keeping memory out of the prefix (yellow) lower the slope.

## 9.4 Choosing the compaction threshold

Figure 9 sweeps the buffer from 5% to 80% of the window. For each session we solved Equation 3 with values measured on the baseline run (mean growth $g$, calls per turn $k$, effective input price $\gamma$, post-compaction level $a$ and mean summary size $s$).

The model's optimum falls within one sweep step of the cheapest setting for both sessions (Table 8). The coding curve is flat between 30% and 60% because there compaction count and timing trade off. It is also uneven at small buffers because the number of compactions is a whole number and depends on where the last tooth ends. Beyond the optimum, cost rises quickly: at an 80% buffer the coding session compacts 29 times and costs 65% more than the default. Below about 10% the in-turn peak reaches 193k, which leaves only 7k of room under a 200k window, less than the 19k largest in-turn growth.

The banking session's default 20% buffer is far from its cost optimum (45% dearer), because its fixed prefix is small and its post-compaction context is cheap to rebuild. Compacting at 38k instead of 102k means seven summaries instead of two. Whether that is acceptable is a quality question that the cost model cannot answer; the post-compaction dip of Section 7.4, measured at each sweep point, is how we intend to answer it.

**Table 8:** Model prediction against the sweep. $\gamma$ is the effective input price at the measured cache retention.

| Session | $g$ | $k$ | $\gamma$ | $a$ | $\Theta^\star$ (buffer) | Sweep minimum (buffer) | Cost at minimum vs default |
|---|---|---|---|---|---|---|---|
| Coding | 7.2k | 3.00 | 0.194 | 33.2k | 97k (51%) | 100k (50%) | −5.9% |
| Banking | 3.1k | 2.66 | 0.195 | 12.4k | 41k (68%) | 38k (70%) | −31.1% |

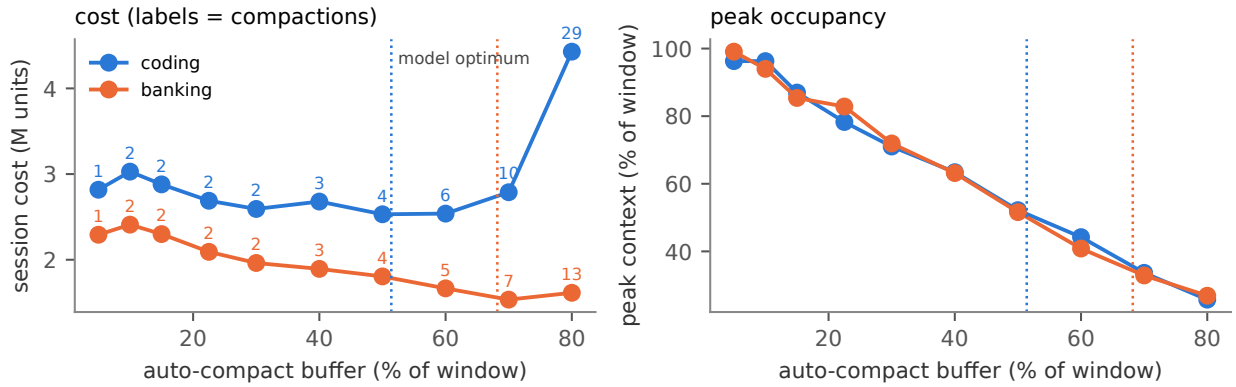



**Figure 9:** Compaction threshold sweep. Left: session cost against buffer size. Labels give the number of compactions, and dotted lines mark $\Theta^\star$ from Equation 3. Right: peak context. Small buffers let the in-turn peak approach the window.

## 9.5 MLflow round trip

To check the exporter, we wrote an eight-turn session to a local MLflow 3.17 store using MAF-style spans. The spans were chat spans with `gen_ai.usage.*` and `pir.tokens.*` attributes, `execute_tool` spans, one memory write and one compaction span. Session metadata was set with `update_current_trace`. The exporter found all eight traces by session and recovered calls, input, cached input, output, tool counts, the memory write, the compaction and the per-category context for each turn. The resulting JSON loads directly into the interactive page. This tests the plumbing only. We have not yet run it on traces from a production MAF deployment, and the evaluation join of Algorithm 1 (line 16) has not yet been part of the round trip.

# 10 Using the Tape

The tape suggests a short list of checks for an agent with a latency target such as the 25-second limit in [2]:

- **Alert on headroom, not occupancy.** A session at 70% occupancy that grows 15k per turn compacts within two turns. One at 85% that grows 2k per turn has seven turns left.
- **Size the buffer from the in-turn peak.** Use the 99th percentile of in-turn growth plus the summary size, measured from the tape, not a fixed percentage.
- **Treat every cache break as a defect with a cause.** The volume panel shows each break. The event strip usually names the cause: a compaction, a memory write in the prefix, a per-request value in the system prompt. Only compaction breaks are unavoidable.
- **Budget tool results.** They are the fastest-growing category. Return summaries or handles and let the agent page through large outputs.
- **Shrink the floor.** The fixed prefix is paid on every call of every turn and sets the post-compaction level. Every token removed with PIR is removed from each tooth of the sawtooth.
- **Watch each agent against its own window.** In a multi-agent session a small router or sub-agent can be the first to compact. Read its lane, not the primary one.
- **Read failures against the columns above them.** A failure cluster under tall columns is a pressure problem; one just after a dashed line is a compaction problem; one under a large red band is a tool-result problem. Each has a different fix.
- **Tune the threshold with Equation 3, then check quality.** Use the square-root rule as a starting point, then compare the post-compaction dip and family fail rates at the new threshold against the old one before adopting it.
- **Change the prompt through the loop.** Run every prompt change on the same suite as the champion, read the diff tape, and promote only through the decision rule of Section 8.4. Keep an A/A run current; when it widens, the suite needs more scenarios or repeats before any comparison means anything.

# 11 Limitations

The sessions are synthetic. Their parameters are plausible but not measured, and real sessions have bursts, topic changes and user pauses that expire the cache (typically after minutes of inactivity), which the model ignores. The cache model is simple: a fixed 97% reuse and a full miss on any prefix change, with no minimum prefix and no per-key rate limits. The compaction call is assumed not to hit the cache, which overstates its cost when a provider keeps the prefix. Cost uses fixed weights, and the output weight is assumed. Answer quality is not measured, so the ablations show what each policy saves, not what it costs in quality. Category counts require a tokeniser in the service. The exporter's rules for recognising memory tools and compaction spans are naming conventions that a given deployment may need to adjust.

The evaluation tape is a design, illustrated on synthetic outcomes; it has not been run on a real agent, and the evaluation panels of Figures 1 and 4 show nothing about how often real agents fail. LLM judges are themselves models, with position, length and self-preference biases [27]; their scores move when the judge or its prompt changes, which is why the record pins both. Rebuilding the context a judge should see is exact only if the trace holds the message text or the agent's context assembler can be replayed. Sampling judges lowers cost but leaves gaps, and a fail rate is only as good as its coverage. Multi-turn judges in MLflow are marked experimental at the time of writing [18].

The comparison loop of Section 8 is illustrated on a synthetic suite whose per-version effects we chose so that each decision path occurs; it shows that the procedure runs and how it reads, not how often real prompt changes help. A suite only measures what its scenarios contain: a prompt can improve on the suite and get worse on traffic it does not cover, which is why production failures must keep flowing into it. Scripted user turns cannot react to a different answer, and a simulated user adds its own variance and bias. Tool fixtures freeze the world, so a version that takes a new, better path meets a "not available" result. Replay mode understates compounding effects. The bootstrap treats scenarios as exchangeable, which a hand-built suite is not. The decision rule controls error for one comparison at a time; over many rounds some false promotions will occur, which the held-out split and fresh-repeat confirmation reduce but do not remove. The regression gate compares failing turns per run with the largest change in one A/A run, a heuristic rather than a test. Weights in $Q$ are a product choice, and a different weighting can reorder versions.

The changes attributed to version 1.2 (Sections 3.2 and 3.3) are reconstructed from the interactive page that implements them, because the 1.2 report file was not available when this version was prepared.

# 12 Future Work

- Export real sessions from the team's MLflow server and repeat Tables 6–8 with measured parameters.
- Run the evaluation tape on those sessions with PIR-derived code checks and two or three judges, and measure the pressure gap and post-compaction dip for real.
- Choose $\Theta$ on cost and quality together: extend Equation 2 with a per-compaction quality penalty estimated from the post-compaction dip.
- Add latency to the tape: time to first token against uncached input per call, to connect cache breaks to the response-time target.
- Track the cacheable prefix over time per Prompt Stack envelope, joining the tape with the static analysis of [1].
- Run the comparison loop on the team's agent with a suite built from real sessions, and report how often promoted versions hold up in production.
- Add replay-mode screening to the exporter, so a candidate can be judged on the champion's recorded contexts before any live run.
- Compare sessions: overlay many tapes as a distribution (median and quantile bands of occupancy and fail rate by turn) to see a deployment's typical trajectory.

# 13 Conclusion

Context in a multi-turn agent is a stock that is replenished every turn and paid for on every call. Drawn as a coloured histogram over turns, with events and token volumes underneath, it shows where cost and latency come from. In our sessions that is resent input, not output. Within it, the expensive part is the uncached tokens that follow compactions and memory writes, and the fastest-growing category is tool results. Three fixes follow directly: keep session-scoped content out of the prefix, budget tool results, and set the compaction threshold with a square-root rule bounded by the in-turn peak. The evaluation tape adds the other half of each decision: whether answers held up as the window filled and after it was compacted, read on the same axis and aggregated so that hundreds of checks stay legible. Run on a fixed multi-turn suite, the same record compares prompt versions: a rubric, a paired interval calibrated by an A/A run, and a diff tape turn prompt changes into a loop that keeps a version only when it is better, or cheaper and no worse. The instrumentation needed is small: a session identifier, a token count per category on spans that frameworks already emit, and evaluation results written back to the same traces.

# Artefacts

- Context Tape page: interactive version of Figures 1–2 with both synthetic sessions, replay, tooltip, data table, per-agent lanes and a JSON loader.
- `context_tape_export.py`: MLflow session exporter (Algorithm 1).
- `sim.py`, `analyze.py`, `figs.py`: workload generator, policies, metrics and figures for Section 9.
- `fig1new.py`, `evaldata.py`, `figeval.py`: Figure 1 redrawn with the evaluation tape, the synthetic check results, and the drill-down of Figure 4.
- `compare.py`: compares prompt versions from schema-1.5 tape records (rubric, A/A noise, bootstrap interval, gates, decision, one extension), and writes `compare_result.json`. It works on any records with a `run` block.
- `gen_suite.py`, `figcompare.py`: the illustrative suite of Section 8.7 (880 runs) and Figures 5 and 6.

# Version History

| | |
|---|---|
| 1.0 (October 2026) | First version: record, visual encoding, metrics, cost model, OTel/MLflow export, synthetic evaluation. |
| 1.2 (October 2026) | Framework-neutral categories with an `unattributed` band; schema 1.3 with one record per model call (agent, model, window); a tape per agent and model. *Reconstructed*: the 1.2 report file was not available when 1.3 was prepared, so these changes are written up here from the Context Tape page that implements them. |
| 1.3 (October 2026) | Evaluation tape (Section 7): sources, collection through MLflow assessments and OpenTelemetry evaluation events, a three-level encoding for many checks, evaluation metrics, an illustrative figure, and schema 1.4. Figure 1 redrawn as the complete tape, with the evaluation tape under the volume panel; data path redrawn with evaluators. Evaluation checks added to Section 10. Companion report retitled *PIR*. References [17, 18, 27] added. |
| 1.4 (October 2026) | Comparing prompt versions (Section 8): fixed multi-turn suite with tool fixtures and dev/held-out split, replay and live modes, a three-layer rubric, a paired bootstrap decision rule with an A/A run and one extension, the diff tape (Figure 5), the champion–challenger loop (Algorithm 2) and an illustrative run (Figure 6, Table 4). Schema 1.5 adds the `run` block. `compare.py` added. Companion reference updated to PIR v2.4. References [19, 20, 21, 22, 23, 24, 25, 26] added. |

# A Record Schema

One JSON object per session; the page also accepts a list of sessions. Schema 1.3 adds `primary_agent` and `calls`; schema 1.4 adds `evals`, `eval_summary` and `session_evals`; schema 1.5 adds the `run` block for suite runs (Section 8.1). Older files load unchanged.

```
{ "schema": "1.5", "session": "dev-10.v4.r3", "window": 128000, "autocompact_buffer": 25600,
  "primary_agent": "assistant",
  "run": { "prompt_name": "coding-agent", "prompt_version": "v4", "parent_version": "v3",   // only for suite runs
           "prompt_hash": "sha256:9f2c...", "change": "carry user facts and open tasks through compaction",
           "suite": "coding-suite@3", "scenario": "dev-10", "split": "dev", "repeat": 3, "mode": "live",
           "agent_model": "large-model", "judge": "judge-model@prompt-v3", "judge_coverage": 1.0 },
  "turns": [
    { "turn": 6, "ts": 1791736701123, "trace_id": "tr-3b7d...",
      "context": { "instructions": 3000, "tool_definitions": 9000, "memory": 800,
                   "summary": 2200, "conversation": 4100, "tool_results": 14900 },  // or {"unattributed": n}
      "input_tokens": 68960, "cached_input_tokens": 55168, "output_tokens": 600,
      "llm_calls": 2, "peak": 35230, "tools": { "kb_search": 2 },
      "events": [ "compaction" ],                       // memory_read, memory_write
      "compaction": { "before": 36730, "after": 15000 },
      "eval_summary": {                                 // one entry per family, always present
        "pir_hard_rules": { "run": 60, "fail": 1, "worst": 0.0, "failing": ["ask_before_destructive"] },
        "quality":        { "run": 18, "fail": 0, "worst": 0.81 },
        "context_health": { "run": 0 } },               // judge sampled out: missing, not passed
      "evals": [                                        // non-pass results in full
        { "name": "ask_before_destructive", "family": "pir_hard_rules", "source": "code",
          "value": 0.0, "threshold": 1.0, "label": "fail", "span_id": "sp-91c...",
          "explanation": "ran `rm -rf build/` without confirmation" } ] } ],
  "calls": [
    { "turn": 6, "agent_id": "router", "model_id": "small-model", "window": 32000,
      "categories": { "instructions": 900, "conversation": 1200 },
      "input_tokens": 2100, "cached_input_tokens": 1800, "output_tokens": 40 },
    { "turn": 6, "agent_id": "assistant", "model_id": "large-model", "window": 128000,
      "categories": { "instructions": 3000, "tool_definitions": 9000, "memory": 800, "summary": 2200,
                      "conversation": 4100, "tool_results": 14900 },
      "input_tokens": 34030, "cached_input_tokens": 27584, "output_tokens": 280 } ],
  "session_evals": [ { "name": "conversation_completeness", "source": "llm_judge",
      "judge": "judge-model@prompt-v3", "value": 0.9, "label": "pass" } ] }
```

# References


[1] A. R. Sharma. PIR: A Prompt Intermediate Representation for Parsing, Analysing, Re-stringing and Refactoring LLM Prompts. Technical report, v2.4, October 2026.

[2] A. R. Sharma. Hops, Not Tokens: latency of a three-tier agent chat solution. Technical report, October 2026.

[3] Anthropic. Claude Code documentation: slash commands (/context, /compact). https://docs.claude.com/en/docs/claude-code/slash-commands

[4] Microsoft. Microsoft Agent Framework. https://github.com/microsoft/agent-framework

[5] OpenTelemetry. Semantic conventions for generative AI systems. https://opentelemetry.io/docs/specs/semconv/gen-ai/

[6] MLflow. Tracing for GenAI applications. https://mlflow.org/docs/latest/genai/tracing/

[7] MLflow. Track users and sessions (trace metadata mlflow.trace.session); OTel session.id grouping, issue #19519. https://github.com/mlflow/mlflow

[8] OpenAI. Prompt caching guide. https://developers.openai.com/api/docs/guides/prompt-caching

[9] E. Lumer et al. Don't Break the Cache. 2026. arXiv:2601.06007.

[10] Y. Ji. Context Engineering for AI Agents: Lessons from Building Manus. 2025. https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus

[11] Anthropic. Effective context engineering for AI agents. 2025. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

[12] C. Packer et al. MemGPT: Towards LLMs as Operating Systems. 2023. arXiv:2310.08560.

[13] N. F. Liu et al. Lost in the Middle: How Language Models Use Long Contexts. TACL, 2024. arXiv:2307.03172.

[14] K. Hong, A. Troynikov, J. Huber. Context Rot. Chroma Research, 2025. https://www.trychroma.com/research/context-rot

[15] Y. Du et al. Context Length Alone Hurts LLM Performance Despite Perfect Retrieval. Findings of EMNLP, 2025. arXiv:2510.05381.

[16] F. W. Harris. How Many Parts to Make at Once. *Factory, The Magazine of Management*, 10(2):135–136, 1913.

[17] OpenTelemetry. Semantic conventions for generative AI events: gen_ai.evaluation.result. https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-events/

[18] MLflow. Evaluating GenAI applications: assessments and feedback (log_feedback), LLM judges and multi-turn (session) evaluation. https://mlflow.org/docs/latest/genai/

[19] MLflow. Conversation simulation (mlflow.genai.simulators.ConversationSimulator; experimental in 3.10). https://mlflow.org/docs/latest/genai/eval-monitor/running-evaluation/conversation-simulation/

[20] MLflow. Prompt registry: manage prompt lifecycles with aliases. https://mlflow.org/docs/latest/genai/prompt-registry/manage-prompt-lifecycles-with-aliases/

[21] MLflow. Optimize prompts (mlflow.genai.optimize_prompts). https://mlflow.org/docs/latest/genai/prompt-registry/optimize-prompts/

[22] S. Yao, N. Shinn, P. Razavi, K. Narasimhan. $\tau$-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. 2024. arXiv:2406.12045.

[23] C. Yang et al. Large Language Models as Optimizers. ICLR, 2024. arXiv:2309.03409.

[24] O. Khattab et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. ICLR, 2024. arXiv:2310.03714.

[25] M. Yuksekgonul et al. TextGrad: Automatic "Differentiation" via Text. 2024. arXiv:2406.07496.

[26] L. A. Agrawal et al. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. 2025. arXiv:2507.19457.

[27] L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS Datasets and Benchmarks, 2023. arXiv:2306.05685.